# Anomaly Detection using Clustering Algorithms

This notebook performs anomaly detection on network-intrusion data using three unsupervised c algorithms:

- KMeans
- Agglomerative Clustering
- DBSCAN

The data is taken from the existing `AnamolyDetection/data` folder. The `class` label is **nc for clustering**; it is used only after clustering to evaluate how well each algorithm separated nor anomalous traffic.

## 1. Imports and Settings

In [1]:
```
import os
from pathlib import Path
import warnings

os.environ.setdefault("MPLCONFIGDIR", "/tmp")
warnings.filterwarnings("ignore", category=FutureWarning)

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from IPython.display import display
from scipy.cluster.hierarchy import dendrogram, linkage

from sklearn.cluster import AgglomerativeClustering, DBSCAN, KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import (
    calinski_harabasz_score,
    ConfusionMatrixDisplay,
    accuracy_score,
    classification_report,
    confusion_matrix,
    davies_bouldin_score,
    f1_score,
    precision_recall_fscore_support,
    silhouette_score,
)
from sklearn.neighbors import NearestNeighbors
from sklearn.preprocessing import StandardScaler

RANDOM_STATE = 42
SAMPLE_PER_CLASS = 1500
BW_HATCHES = ["///", "\\\\\\", "xx", "..", "++", "oo"]
BW_COLORS = ["white", "#E6E6E6", "#BFBFBF", "#8C8C8C", "#666666", "#D9D9D
BW_MARKERS = ["o", "s", "^", "D", "x", "P", "*"]
BW_LINESTYLES = ["-", "--", "-.", ":"]
```

```
sns.set_theme(style="whitegrid")
plt.rcParams.update({
    "figure.dpi": 120,
    "axes.edgecolor": "black",
    "axes.labelcolor": "black",
    "xtick.color": "black",
    "ytick.color": "black",
    "text.color": "black",
})
pd.set_option("display.max_columns", 120)
pd.set_option("display.max_colwidth", 120)
```

## 1.1 Helper Functions

In [2]:

```
def add_bar_labels(ax, horizontal=False):
    for patch in ax.patches:
        value = patch.get_width() if horizontal else patch.get_height()
        if np.isnan(value):
            continue
        if horizontal:
            ax.annotate(
                f"{value:,.0f}",
                (value, patch.get_y() + patch.get_height() / 2),
                ha="left",
                va="center",
                xytext=(4, 0),
                textcoords="offset points",
                fontsize=9,
            )
        else:
            ax.annotate(
                f"{value:,.0f}",
                (patch.get_x() + patch.get_width() / 2, value),
                ha="center",
                va="bottom",
                xytext=(0, 3),
                textcoords="offset points",
                fontsize=9,
            )


def plot_count_bar(counts, title, xlabel="", ylabel="", horizontal=False,
    plot_counts = counts.copy()
    if top_n is not None:
        plot_counts = plot_counts.head(top_n)
    fig, ax = plt.subplots(figsize=(9, max(4, 0.35 * len(plot_counts))) i
    colors = [BW_COLORS[i % len(BW_COLORS)] for i in range(len(plot_count
    if horizontal:
        bars = ax.barh(plot_counts.index.astype(str), plot_counts.values,
        ax.invert_yaxis()
        ax.set_xlabel(xlabel)
        ax.set_ylabel(ylabel)
        ax.margins(x=0.15)
    else:
        bars = ax.bar(plot_counts.index.astype(str), plot_counts.values,
        ax.set_xlabel(xlabel)
        ax.set_ylabel(ylabel)
        ax.tick_params(axis="x", rotation=25)
```

```
        ax.margins(y=0.15)
    for idx, bar in enumerate(bars):
        bar.set_hatch(BW_HATCHES[idx % len(BW_HATCHES)])
    ax.set_title(title)
    add_bar_labels(ax, horizontal=horizontal)
    plt.tight_layout()
    plt.show()


def get_data_root():
    candidates = [
        Path("AnamolyDetection/data"),
        Path("../data"),
        Path.cwd() / "AnamolyDetection" / "data",
    ]
    return next(path for path in candidates if path.exists())


def stratified_sample(data, label_col, n_per_class=SAMPLE_PER_CLASS):
    parts = []
    for label_value, group in data.groupby(label_col):
        parts.append(group.sample(min(len(group), n_per_class), random_st
    return pd.concat(parts).sample(frac=1, random_state=RANDOM_STATE).res


def preprocess_features(data, label_col):
    feature_data = data.drop(columns=[label_col]).copy()
    categorical_cols = feature_data.select_dtypes(include=["object", "str
    processed = pd.get_dummies(feature_data, columns=categorical_cols, dr
    processed = processed.apply(pd.to_numeric, errors="coerce").replace([
    scaler = StandardScaler()
    scaled = scaler.fit_transform(processed)
    return processed, scaled, scaler


def map_clusters_to_anomaly(cluster_labels, y_true):
    mapped = np.zeros_like(y_true, dtype=int)
    mapping_rows = []
    for cluster_id in sorted(pd.Series(cluster_labels).unique()):
        mask = cluster_labels == cluster_id
        if cluster_id == -1:
            mapped_label = 1
            rule = "DBSCAN noise -> anomaly"
        else:
            counts = pd.Series(y_true[mask]).value_counts()
            mapped_label = int(counts.idxmax())
            rule = "majority label for evaluation"
        mapped[mask] = mapped_label
        mapping_rows.append(
            {
                "cluster": cluster_id,
                "mapped_prediction": "anomaly" if mapped_label == 1 else
                "cluster_size": int(mask.sum()),
                "true_anomaly_rate": float(y_true[mask].mean()) if mask.s
                "rule": rule,
            }
        )
    return mapped, pd.DataFrame(mapping_rows)
```

```python
def internal_cluster_metrics(X, cluster_labels, ignore_noise=True):
    labels = np.asarray(cluster_labels)
    if ignore_noise:
        valid_mask = labels != -1
        X_eval = X[valid_mask]
        labels_eval = labels[valid_mask]
    else:
        X_eval = X
        labels_eval = labels

    unique_labels = np.unique(labels_eval)
    if X_eval.shape[0] < 2 or len(unique_labels) < 2:
        return {
            "Silhouette Score": np.nan,
            "Davies-Bouldin Index": np.nan,
            "Calinski-Harabasz Index": np.nan,
            "Metric Notes": "Not enough non-noise clusters",
        }

    return {
        "Silhouette Score": silhouette_score(X_eval, labels_eval),
        "Davies-Bouldin Index": davies_bouldin_score(X_eval, labels_eval)
        "Calinski-Harabasz Index": calinski_harabasz_score(X_eval, labels
        "Metric Notes": "Noise ignored" if ignore_noise and np.any(labels
    }


def evaluate_clustering(name, cluster_labels, y_true, X_for_internal_metr
    y_pred, mapping_df = map_clusters_to_anomaly(cluster_labels, y_true)
    precision, recall, f1, _ = precision_recall_fscore_support(
        y_true,
        y_pred,
        average="weighted",
        zero_division=0,
    )
    result = {
        "Model": name,
        "Clusters": len(set(cluster_labels)) - (1 if -1 in cluster_labels
        "Noise Points": int((cluster_labels == -1).sum()),
        "Accuracy": accuracy_score(y_true, y_pred),
        "Weighted Precision": precision,
        "Weighted Recall": recall,
        "Weighted F1": f1,
        "Anomaly F1": f1_score(y_true, y_pred, zero_division=0),
    }
    result.update(internal_cluster_metrics(X_for_internal_metrics, cluste

    print(f"{name} cluster-to-label mapping")
    display(mapping_df)

    print(f"{name} classification report")
    display(pd.DataFrame(classification_report(y_true, y_pred, target_nam

    cm = confusion_matrix(y_true, y_pred, labels=[0, 1])
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["n
    disp.plot(cmap="Greys", values_format="d", colorbar=False)
    plt.title(f"{name} Confusion Matrix")
    plt.tight_layout()
    plt.show()
    return result, y_pred
```

```
def plot_pca_scatter(pca_df, color_col, title):
    fig, ax = plt.subplots(figsize=(8, 5.5))
    labels = sorted(pca_df[color_col].unique(), key=lambda x: str(x))
    for idx, label_value in enumerate(labels):
        subset = pca_df[pca_df[color_col] == label_value]
        ax.scatter(
            subset["PC1"],
            subset["PC2"],
            s=18,
            facecolors="none" if idx % 2 == 0 else BW_COLORS[idx % len(BW
            edgecolors="black",
            marker=BW_MARKERS[idx % len(BW_MARKERS)],
            linewidths=0.7,
            alpha=0.75,
            label=str(label_value),
        )
    ax.set_title(title)
    ax.set_xlabel("PC1")
    ax.set_ylabel("PC2")
    ax.legend(title=color_col, bbox_to_anchor=(1.02, 1), loc="upper left"
    plt.tight_layout()
    plt.show()


def plot_metric_lines(df, x_col, y_cols, title, ylabel):
    fig, ax = plt.subplots(figsize=(8.5, 4.8))
    for idx, col in enumerate(y_cols):
        ax.plot(
            df[x_col],
            df[col],
            color=BW_COLORS[min(idx + 1, len(BW_COLORS) - 1)] if idx else
            marker=BW_MARKERS[idx % len(BW_MARKERS)],
            linestyle=BW_LINESTYLES[idx % len(BW_LINESTYLES)],
            linewidth=1.8,
            label=col,
        )
    ax.set_title(title)
    ax.set_xlabel(x_col)
    ax.set_ylabel(ylabel)
    ax.legend()
    plt.tight_layout()
    plt.show()
```

## 2. Load Dataset

In [3]:
```
DATA_ROOT = get_data_root()
raw_path = DATA_ROOT / "raw" / "Network_Intrusion_Detection" / "Train_dat

df_raw = pd.read_csv(raw_path)
print("Using dataset:", raw_path.resolve())
print("Raw shape:", df_raw.shape)
display(df_raw.head())
```

```
Using dataset: /home/durgaumadev/security-for-data-science-lab/AnamolyDetecti
data/raw/Network_Intrusion_Detection/Train_data.csv
Raw shape: (25192, 42)
```

| | duration | protocol_type | service | flag | src_bytes | dst_bytes | land | wrong_fragment | urg |
|---|---|---|---|---|---|---|---|---|---|
| **0** | 0 | tcp | ftp_data | SF | 491 | 0 | 0 | 0 | |
| **1** | 0 | udp | other | SF | 146 | 0 | 0 | 0 | |
| **2** | 0 | tcp | private | S0 | 0 | 0 | 0 | 0 | |
| **3** | 0 | tcp | http | SF | 232 | 8153 | 0 | 0 | |
| **4** | 0 | tcp | http | SF | 199 | 420 | 0 | 0 | |

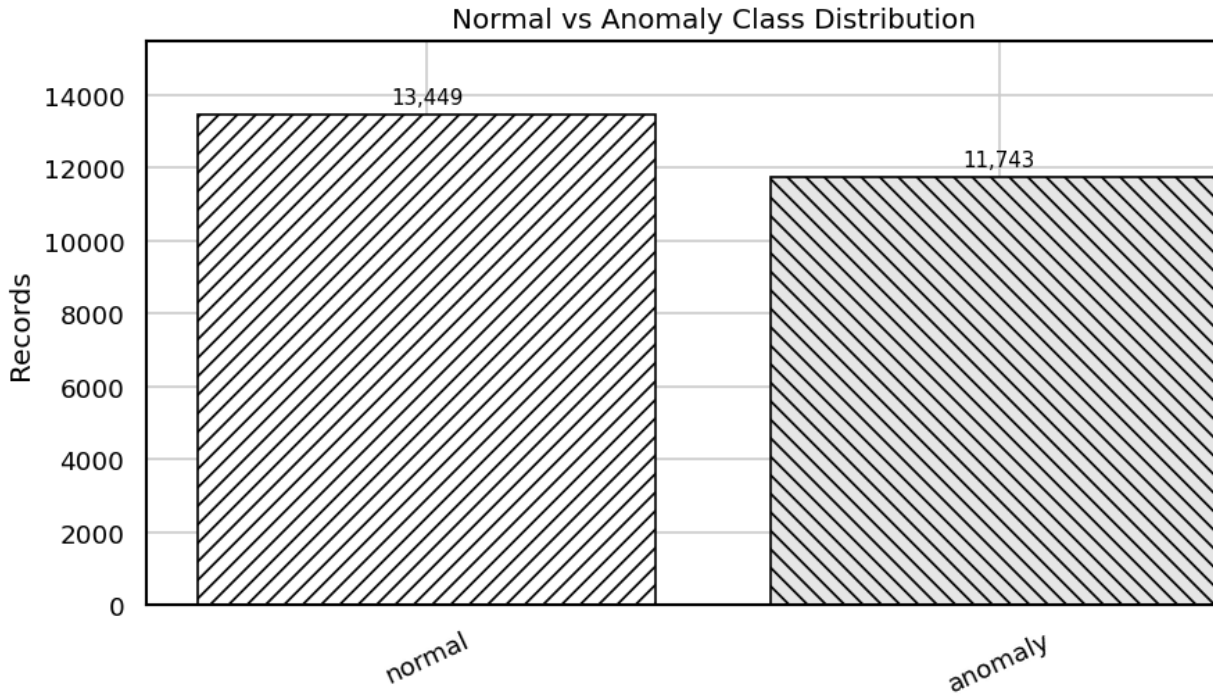
# 3. Exploratory Data Analysis

In [4]:
```
print("Columns:", df_raw.columns.tolist())
print("\nMissing values")
display(pd.DataFrame({
    "missing": df_raw.isna().sum(),
    "missing_pct": (df_raw.isna().mean() * 100).round(2),
}).sort_values("missing", ascending=False).head(15))

class_counts = df_raw["class"].value_counts()
display(class_counts.rename_axis("class").reset_index(name="count"))
plot_count_bar(class_counts, title="Normal vs Anomaly Class Distribution"

for col in ["protocol_type", "flag", "service"]:
    print(f"Top values for {col}")
    display(df_raw[col].value_counts().head(12).rename_axis(col).reset_in
    plot_count_bar(df_raw[col].value_counts(), title=f"Top {col} Values",
```

```
Columns: ['duration', 'protocol_type', 'service', 'flag', 'src_bytes', 'dst_b
'land', 'wrong_fragment', 'urgent', 'hot', 'num_failed_logins', 'logged_in',
'num_compromised', 'root_shell', 'su_attempted', 'num_root', 'num_file_creati
'num_shells', 'num_access_files', 'num_outbound_cmds', 'is_host_login',
'is_guest_login', 'count', 'srv_count', 'serror_rate', 'srv_serror_rate',
'rerror_rate', 'srv_rerror_rate', 'same_srv_rate', 'diff_srv_rate',
'srv_diff_host_rate', 'dst_host_count', 'dst_host_srv_count',
'dst_host_same_srv_rate', 'dst_host_diff_srv_rate', 'dst_host_same_src_port_r
'dst_host_srv_diff_host_rate', 'dst_host_serror_rate', 'dst_host_srv_serror_r
'dst_host_rerror_rate', 'dst_host_srv_rerror_rate', 'class']

Missing values
```

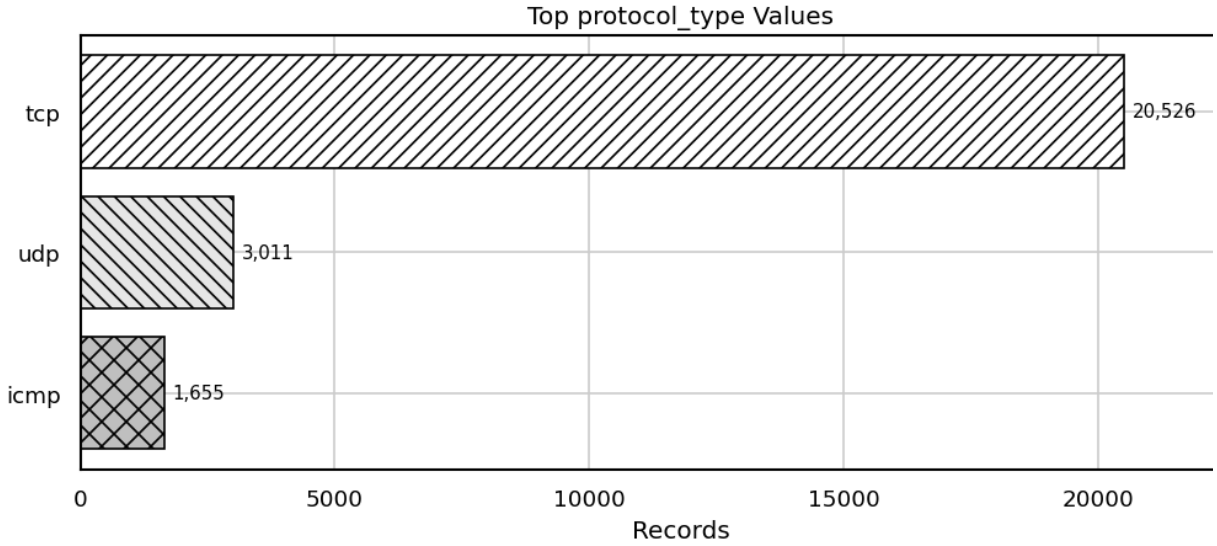
| | missing | missing_pct |
|---|---|---|
| **duration** | 0 | 0.0 |
| **protocol_type** | 0 | 0.0 |
| **service** | 0 | 0.0 |
| **flag** | 0 | 0.0 |
| **src_bytes** | 0 | 0.0 |
| **dst_bytes** | 0 | 0.0 |
| **land** | 0 | 0.0 |
| **wrong_fragment** | 0 | 0.0 |
| **urgent** | 0 | 0.0 |
| **hot** | 0 | 0.0 |
| **num_failed_logins** | 0 | 0.0 |
| **logged_in** | 0 | 0.0 |
| **num_compromised** | 0 | 0.0 |
| **root_shell** | 0 | 0.0 |
| **su_attempted** | 0 | 0.0 |

| | class | count |
|---|---|---|
| **0** | normal | 13449 |
| **1** | anomaly | 11743 |



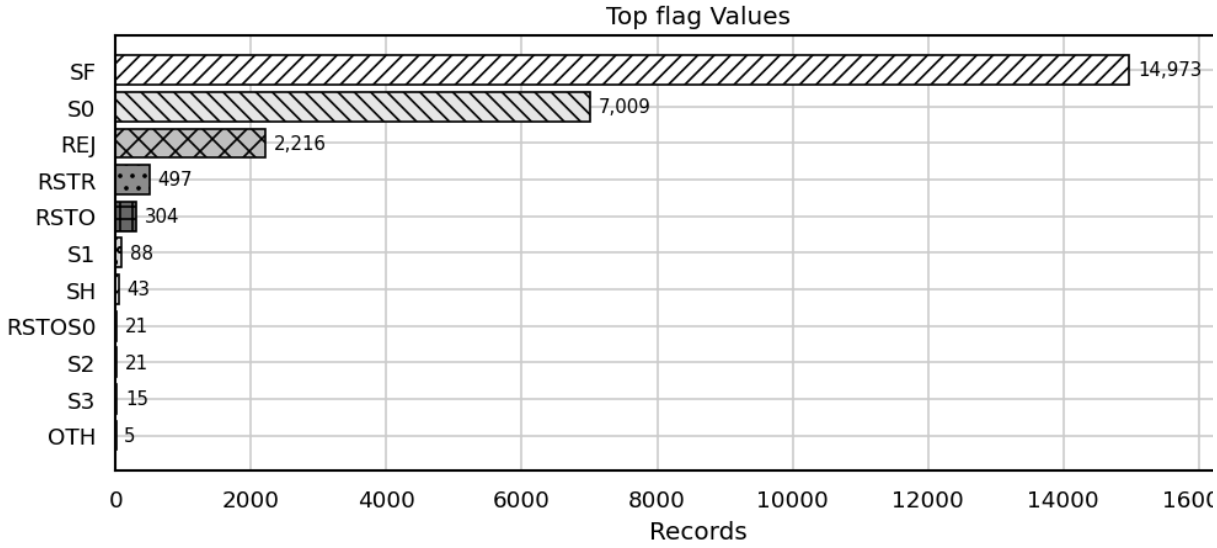

Top values for protocol_type

| | protocol_type | count |
|---|---|---|
| **0** | tcp | 20526 |
| **1** | udp | 3011 |
| **2** | icmp | 1655 |



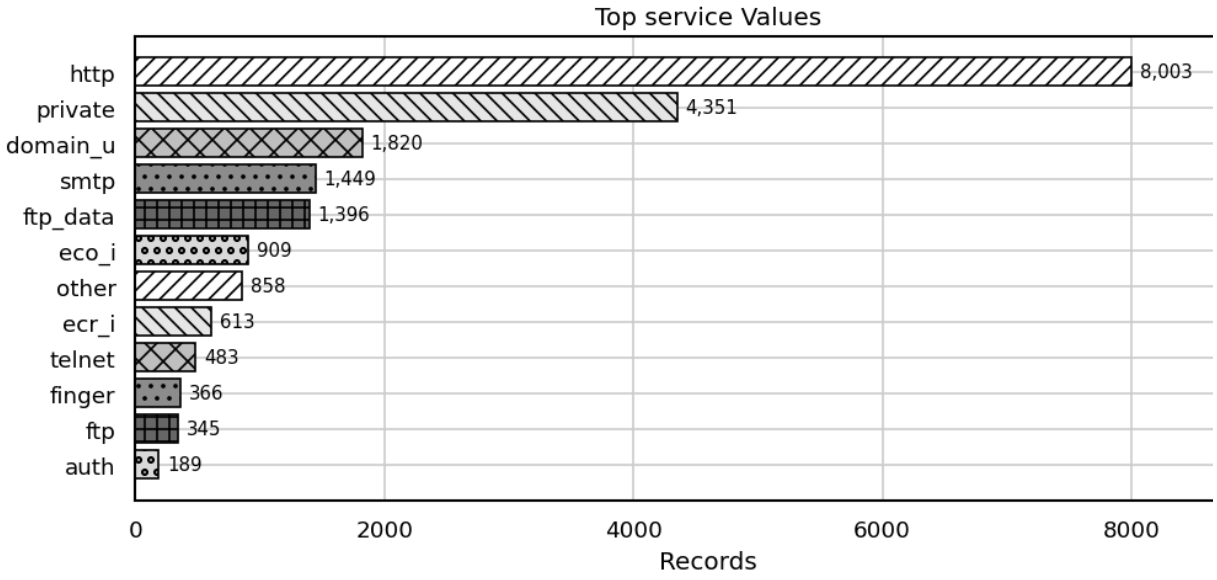

Top values for flag

| | flag | count |
|---|---|---|
| **0** | SF | 14973 |
| **1** | S0 | 7009 |
| **2** | REJ | 2216 |
| **3** | RSTR | 497 |
| **4** | RSTO | 304 |
| **5** | S1 | 88 |
| **6** | SH | 43 |
| **7** | RSTOS0 | 21 |
| **8** | S2 | 21 |
| **9** | S3 | 15 |
| **10** | OTH | 5 |




Top values for service

| | service | count |
|---|---|---|
| 0 | http | 8003 |
| 1 | private | 4351 |
| 2 | domain_u | 1820 |
| 3 | smtp | 1449 |
| 4 | ftp_data | 1396 |
| 5 | eco_i | 909 |
| 6 | other | 858 |
| 7 | ecr_i | 613 |
| 8 | telnet | 483 |
| 9 | finger | 366 |
| 10 | ftp | 345 |
| 11 | auth | 189 |




# 4. Sampling and Preprocessing

In [5]:
```
df = stratified_sample(df_raw, label_col="class", n_per_class=SAMPLE_PER_
print("Sampled shape:", df.shape)
display(df["class"].value_counts().rename_axis("class").reset_index(name=

y_true = (df["class"] != "normal").astype(int).to_numpy()
X_processed, X_scaled, scaler = preprocess_features(df, label_col="class"

print("Processed feature shape:", X_processed.shape)
display(X_processed.head())
```

```
Sampled shape: (3000, 42)
```

| | class | count |
|---|---|---|
| 0 | normal | 1500 |
| 1 | anomaly | 1500 |

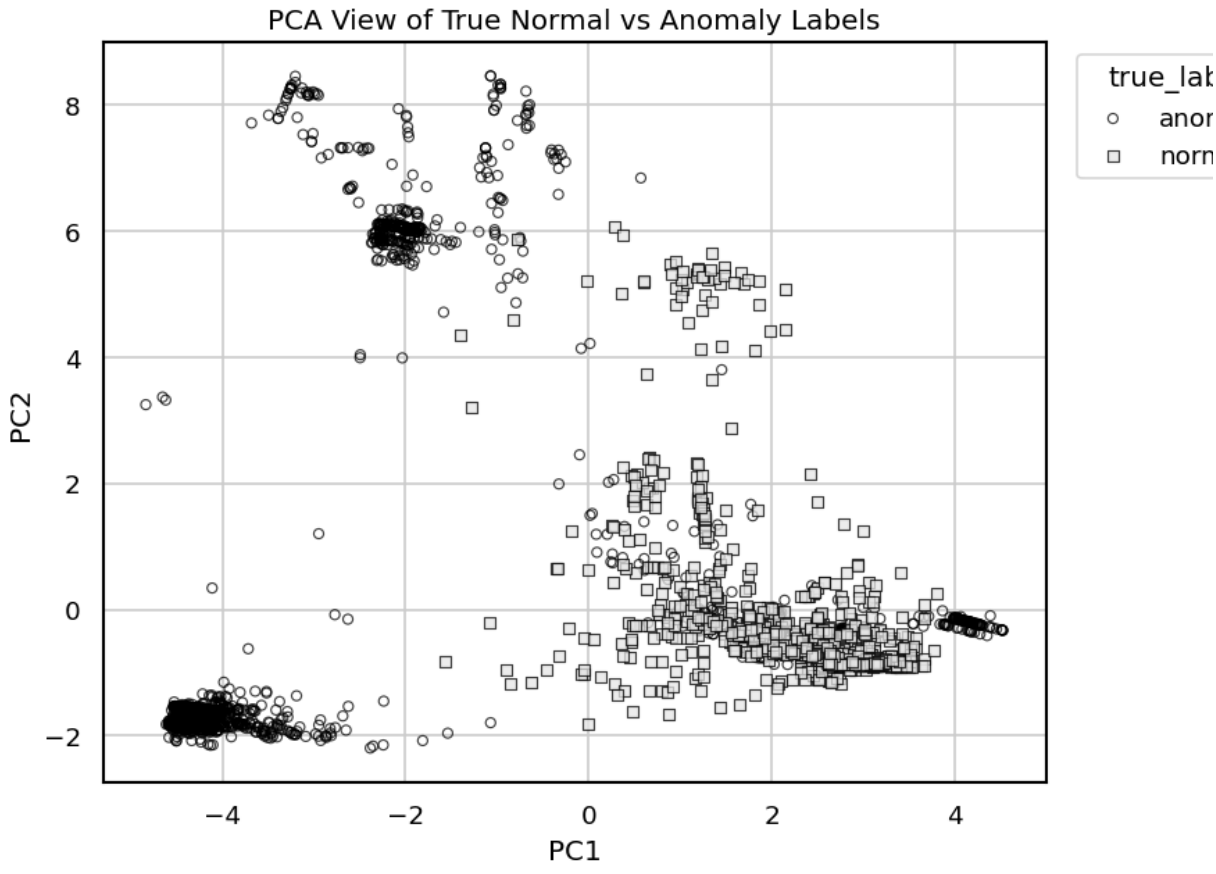
```
Processed feature shape: (3000, 109)
```

| | duration | src_bytes | dst_bytes | land | wrong_fragment | urgent | hot | num_failed_logins | l |
|---|---|---|---|---|---|---|---|---|---|
| **0** | 0 | 2444 | 329 | 0 | 0 | 0 | 0 | 0 | |
| **1** | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | |
| **2** | 0 | 10044 | 0 | 0 | 0 | 0 | 0 | 0 | |
| **3** | 0 | 1032 | 0 | 0 | 0 | 0 | 0 | 0 | |
| **4** | 0 | 163 | 1510 | 0 | 0 | 0 | 0 | 0 | |

## 5. PCA Visualization

In [6]:
```
pca = PCA(n_components=2, random_state=RANDOM_STATE)
X_pca = pca.fit_transform(X_scaled)
pca_df = pd.DataFrame(X_pca, columns=["PC1", "PC2"])
pca_df["true_label"] = np.where(y_true == 1, "anomaly", "normal")

print("Explained variance ratio:", np.round(pca.explained_variance_ratio_
plot_pca_scatter(pca_df, color_col="true_label", title="PCA View of True
```

```
Explained variance ratio: [0.0919 0.0591]
```

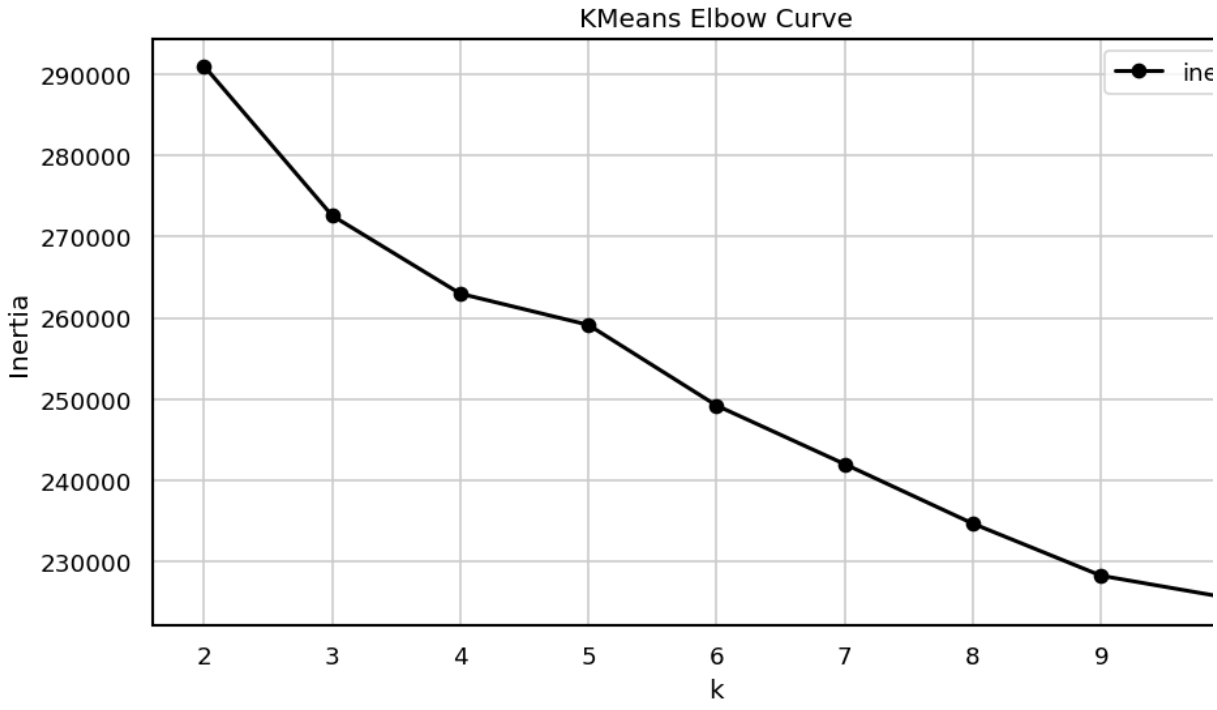


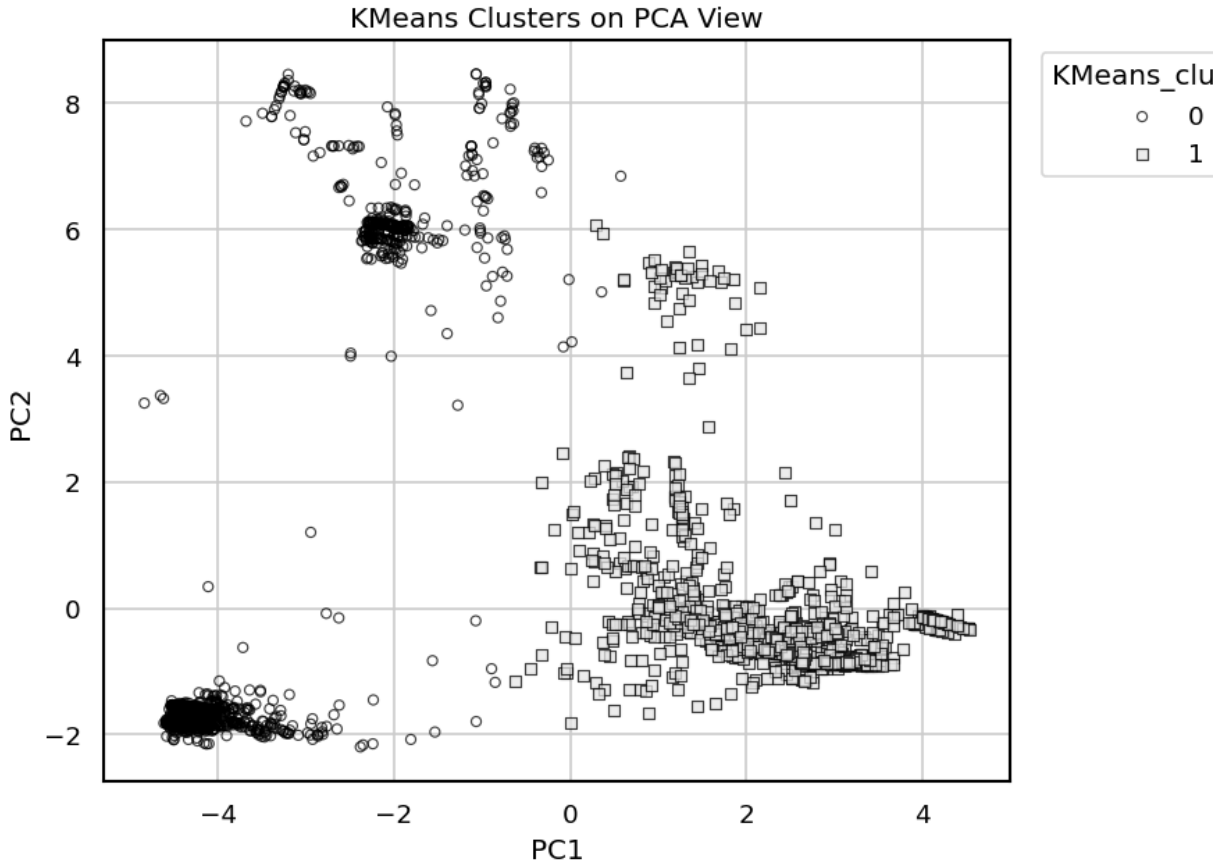

## 6. KMeans Clustering

In [7]:
```
k_values = range(2, 11)
kmeans_rows = []
for k in k_values:
    model = KMeans(n_clusters=k, n_init=20, random_state=RANDOM_STATE)
    labels = model.fit_predict(X_scaled)
    kmeans_rows.append({
```

```
        "k": k,
        "inertia": model.inertia_,
        "silhouette": silhouette_score(X_scaled, labels),
    })

kmeans_curve = pd.DataFrame(kmeans_rows)
display(kmeans_curve.round(4))
plot_metric_lines(kmeans_curve, "k", ["inertia"], "KMeans Elbow Curve", "
plot_metric_lines(kmeans_curve, "k", ["silhouette"], "KMeans Silhouette b

kmeans_model = KMeans(n_clusters=2, n_init=20, random_state=RANDOM_STATE)
kmeans_labels = kmeans_model.fit_predict(X_scaled)
pca_df["KMeans_cluster"] = kmeans_labels.astype(str)
plot_pca_scatter(pca_df, color_col="KMeans_cluster", title="KMeans Cluste

results = []
predictions = {}
kmeans_result, kmeans_pred = evaluate_clustering("KMeans", kmeans_labels,
results.append(kmeans_result)
predictions["KMeans"] = kmeans_pred
```

| | k | inertia | silhouette |
|---|---|---|---|
| **0** | 2 | 291026.8672 | 0.2203 |
| **1** | 3 | 272595.8227 | 0.2444 |
| **2** | 4 | 262986.8678 | 0.1651 |
| **3** | 5 | 259135.8047 | 0.1994 |
| **4** | 6 | 249211.9326 | 0.1817 |
| **5** | 7 | 241998.6411 | 0.2068 |
| **6** | 8 | 234677.1906 | 0.2162 |
| **7** | 9 | 228266.8111 | 0.2105 |
| **8** | 10 | 225509.4613 | 0.1957 |



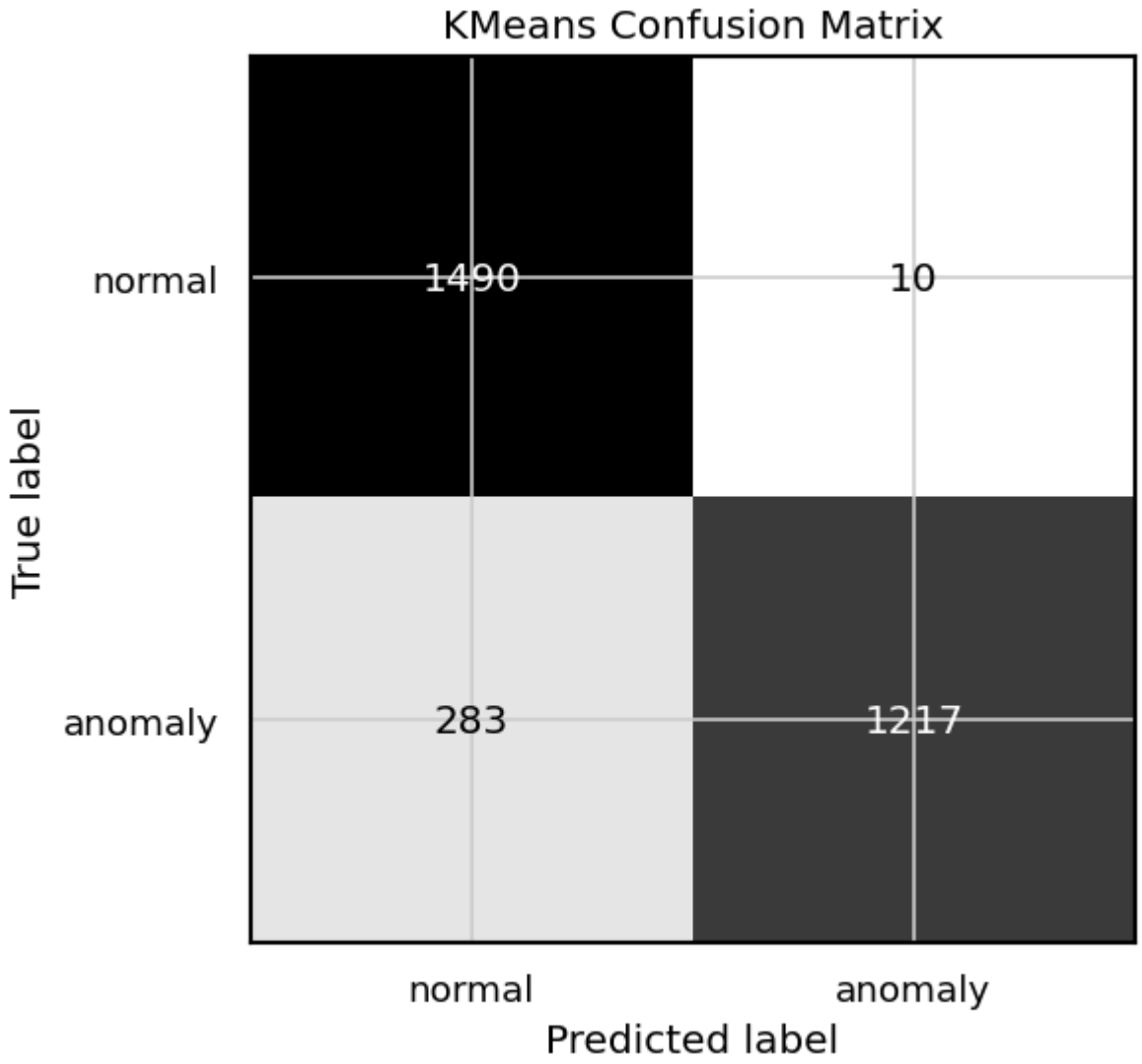




KMeans cluster-to-label mapping

| | cluster | mapped_prediction | cluster_size | true_anomaly_rate | rule |
|---|---|---|---|---|---|
| **0** | 0 | anomaly | 1227 | 0.991850 | majority label for evaluation |
| **1** | 1 | normal | 1773 | 0.159616 | majority label for evaluation |

KMeans classification report

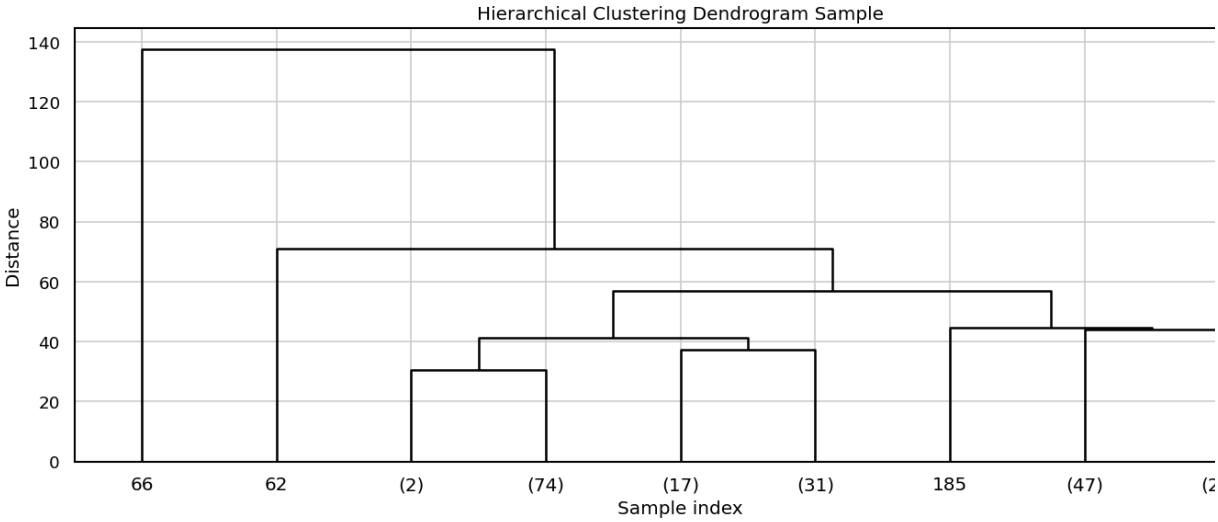
| | precision | recall | f1-score | support |
|---|---|---|---|---|
| **normal** | 0.8404 | 0.9933 | 0.9105 | 1500.0000 |
| **anomaly** | 0.9919 | 0.8113 | 0.8926 | 1500.0000 |
| **accuracy** | 0.9023 | 0.9023 | 0.9023 | 0.9023 |
| **macro avg** | 0.9161 | 0.9023 | 0.9015 | 3000.0000 |
| **weighted avg** | 0.9161 | 0.9023 | 0.9015 | 3000.0000 |

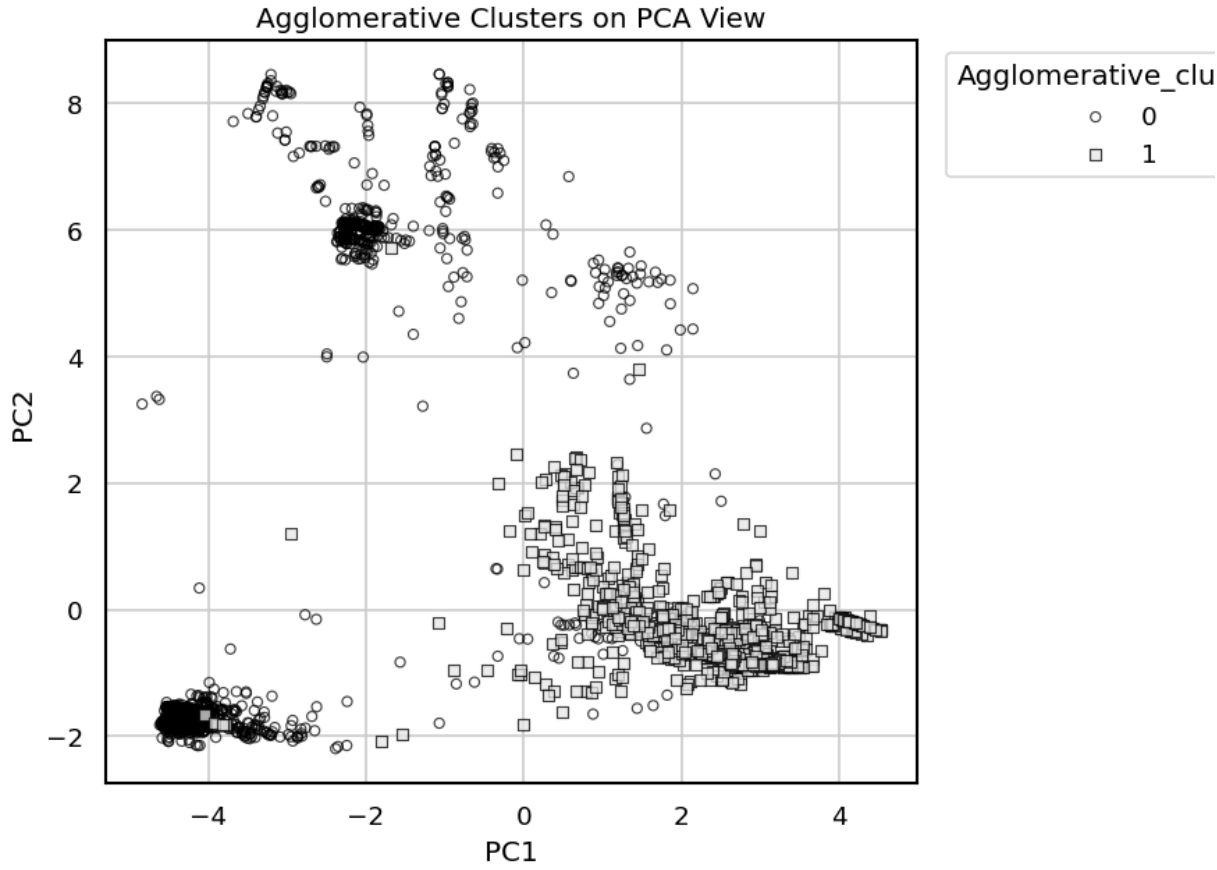



## 7. Agglomerative Clustering

In [8]:
```
dendrogram_sample_size = min(200, X_scaled.shape[0])
rng = np.random.default_rng(RANDOM_STATE)
dendrogram_idx = rng.choice(X_scaled.shape[0], size=dendrogram_sample_siz
linked = linkage(X_scaled[dendrogram_idx], method="ward")

plt.figure(figsize=(12, 5))
dendrogram(linked, truncate_mode="level", p=4, color_threshold=0, above_t
plt.title("Hierarchical Clustering Dendrogram Sample")
plt.xlabel("Sample index")
plt.ylabel("Distance")
plt.tight_layout()
plt.show()
```

```
agg_model = AgglomerativeClustering(n_clusters=2, linkage="ward")
agg_labels = agg_model.fit_predict(X_scaled)
pca_df["Agglomerative_cluster"] = agg_labels.astype(str)
plot_pca_scatter(pca_df, color_col="Agglomerative_cluster", title="Agglom

agg_result, agg_pred = evaluate_clustering("Agglomerative", agg_labels, y
results.append(agg_result)
predictions["Agglomerative"] = agg_pred
```



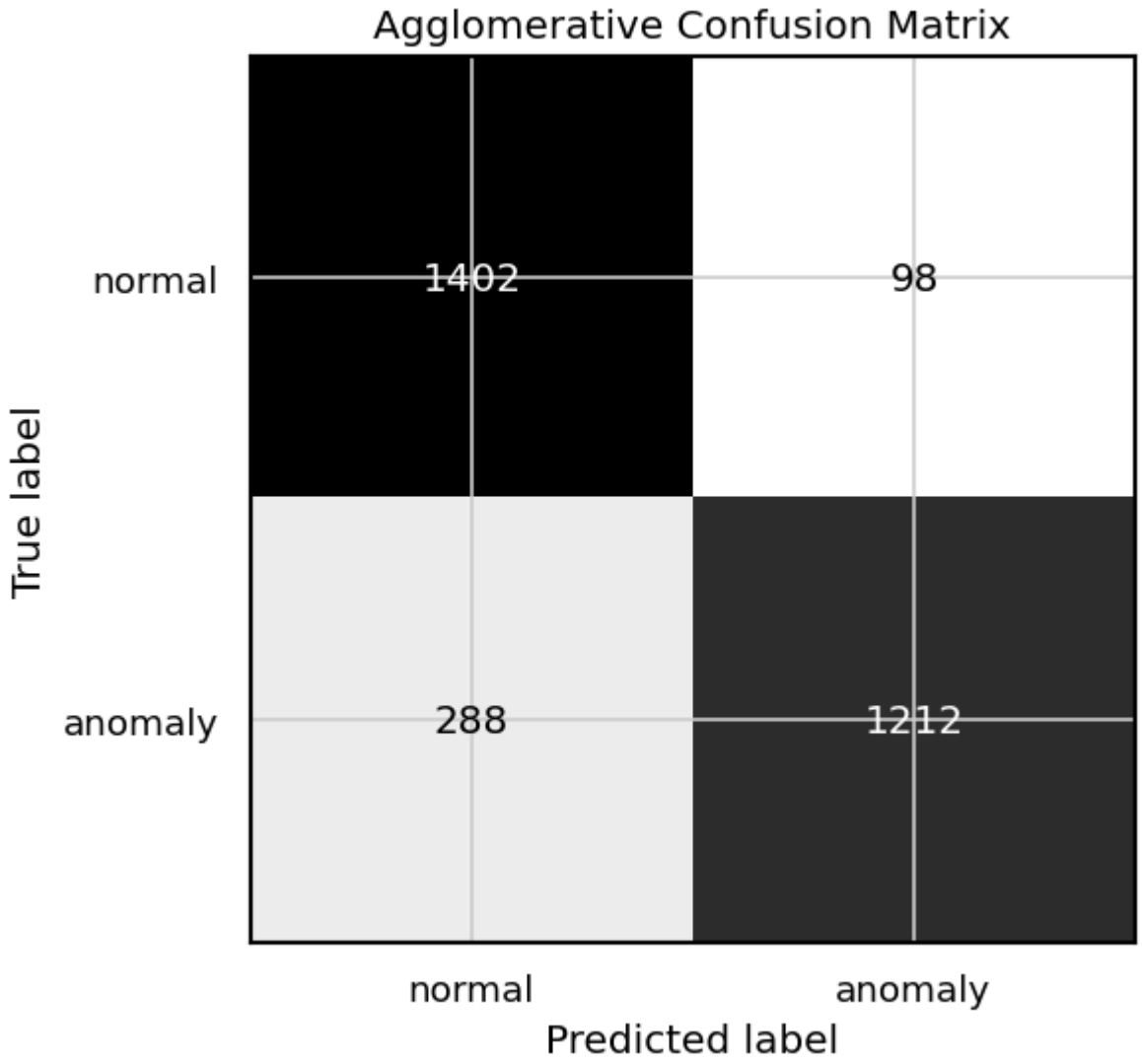




Agglomerative cluster-to-label mapping

| | cluster | mapped_prediction | cluster_size | true_anomaly_rate | rule |
|---|---|---|---|---|---|
| **0** | 0 | anomaly | 1310 | 0.925191 | majority label for evaluation |
| **1** | 1 | normal | 1690 | 0.170414 | majority label for evaluation |

Agglomerative classification report

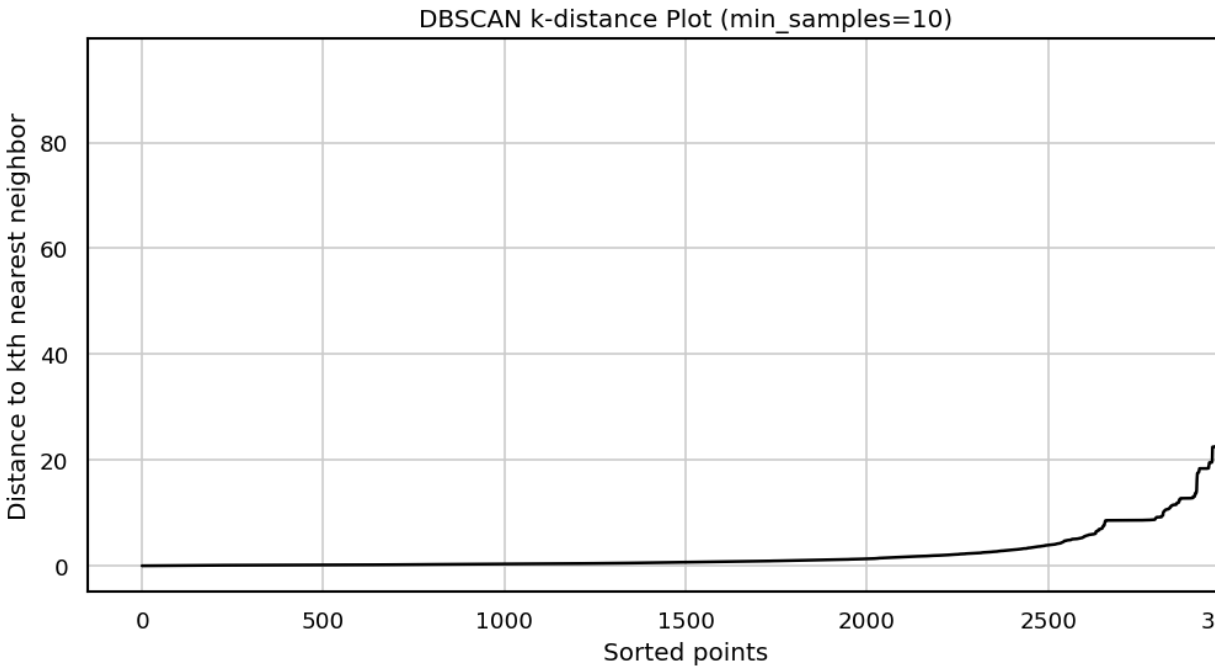
| | precision | recall | f1-score | support |
|---|---|---|---|---|
| **normal** | 0.8296 | 0.9347 | 0.8790 | 1500.0000 |
| **anomaly** | 0.9252 | 0.8080 | 0.8626 | 1500.0000 |
| **accuracy** | 0.8713 | 0.8713 | 0.8713 | 0.8713 |
| **macro avg** | 0.8774 | 0.8713 | 0.8708 | 3000.0000 |
| **weighted avg** | 0.8774 | 0.8713 | 0.8708 | 3000.0000 |



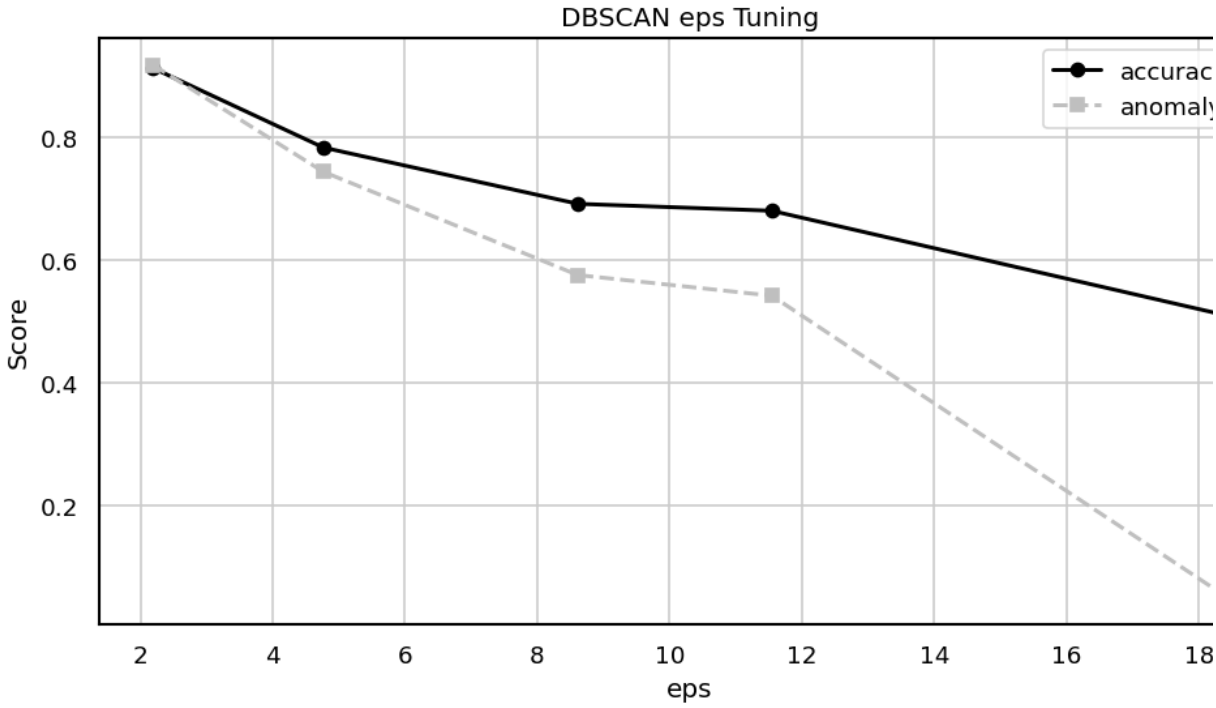

## 8. DBSCAN Hyperparameter Exploration

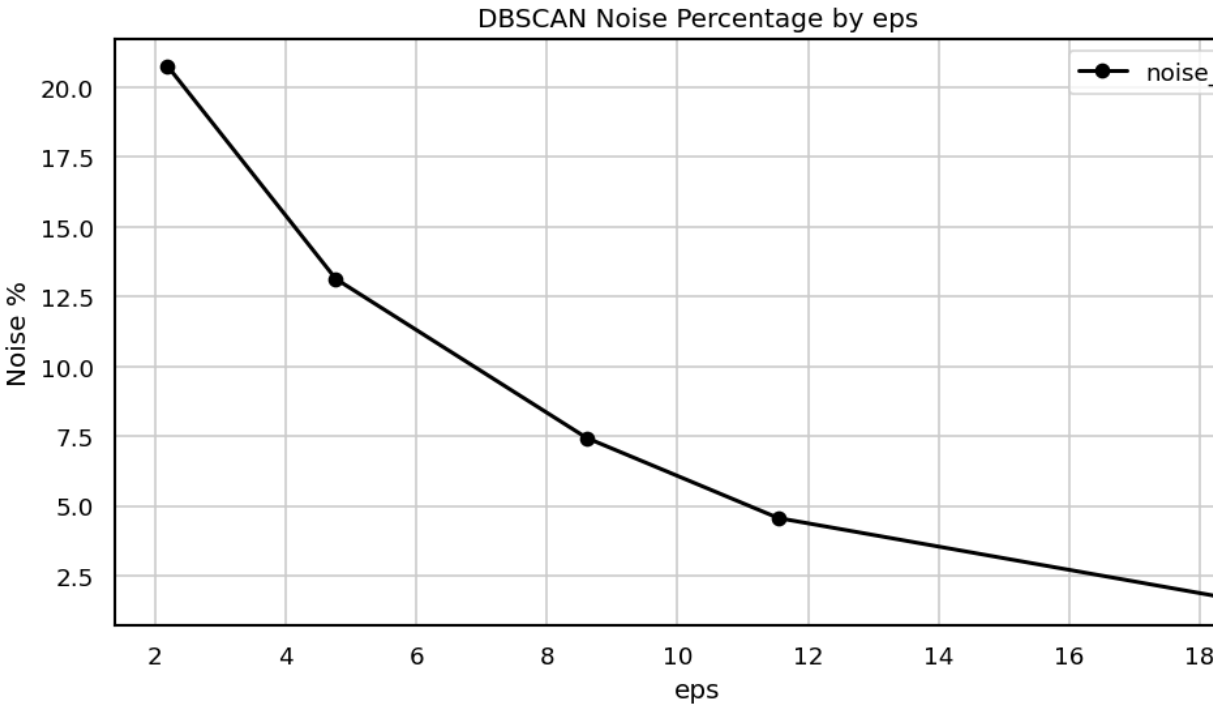
In [9]:
```
min_samples = 10
neighbors = NearestNeighbors(n_neighbors=min_samples)
neighbors.fit(X_scaled)
distances, _ = neighbors.kneighbors(X_scaled)
k_distances = np.sort(distances[:, -1])

plt.figure(figsize=(9, 4.8))
plt.plot(np.arange(len(k_distances)), k_distances, color="black", linewid
plt.title(f"DBSCAN k-distance Plot (min_samples={min_samples})")
plt.xlabel("Sorted points")
plt.ylabel("Distance to kth nearest neighbor")
plt.tight_layout()
plt.show()
```

```
eps_candidates = np.quantile(k_distances, [0.75, 0.85, 0.90, 0.95, 0.98])
dbscan_rows = []
for eps in eps_candidates:
    labels = DBSCAN(eps=float(eps), min_samples=min_samples, n_jobs=-1).f
    pred, _ = map_clusters_to_anomaly(labels, y_true)
    dbscan_rows.append({
        "eps": float(eps),
        "clusters": len(set(labels)) - (1 if -1 in labels else 0),
        "noise_points": int((labels == -1).sum()),
        "noise_pct": (labels == -1).mean() * 100,
        "accuracy": accuracy_score(y_true, pred),
        "anomaly_f1": f1_score(y_true, pred, zero_division=0),
        **internal_cluster_metrics(X_scaled, labels, ignore_noise=True),
    })

dbscan_tuning = pd.DataFrame(dbscan_rows)
display(dbscan_tuning.round(4))
plot_metric_lines(dbscan_tuning, "eps", ["accuracy", "anomaly_f1"], "DBSC
plot_metric_lines(dbscan_tuning, "eps", ["noise_pct"], "DBSCAN Noise Perc
```




| | eps | clusters | noise_points | noise_pct | accuracy | anomaly_f1 | Silhouette Score | Davies-Bouldin Index |
|---|---|---|---|---|---|---|---|---|
| **0** | 2.1874 | 48 | 623 | 20.7667 | 0.9123 | 0.9174 | 0.6983 | 0.3491 |
| **1** | 4.7672 | 41 | 394 | 13.1333 | 0.7827 | 0.7431 | 0.4420 | 0.5087 |
| **2** | 8.6115 | 31 | 223 | 7.4333 | 0.6910 | 0.5750 | 0.4077 | 0.6014 |
| **3** | 11.5436 | 34 | 137 | 4.5667 | 0.6797 | 0.5413 | 0.4765 | 0.5957 |
| **4** | 18.4213 | 1 | 51 | 1.7000 | 0.5090 | 0.0503 | NaN | NaN |




# 9. DBSCAN Clustering

In [ ]:
```
best_eps = dbscan_tuning.sort_values("anomaly_f1", ascending=False).iloc[
print("Selected eps from tuning table:", round(best_eps, 4))

dbscan_model = DBSCAN(eps=float(best_eps), min_samples=min_samples, n_job
dbscan_labels = dbscan_model.fit_predict(X_scaled)
pca_df["DBSCAN_cluster"] = dbscan_labels.astype(str)
plot_pca_scatter(pca_df, color_col="DBSCAN_cluster", title="DBSCAN Cluste

dbscan_result, dbscan_pred = evaluate_clustering("DBSCAN", dbscan_labels,
results.append(dbscan_result)
predictions["DBSCAN"] = dbscan_pred
```

# 10. Model Comparison

In [11]:

```
comparison_df = pd.DataFrame(results)
display(comparison_df.round(4))

print("Internal clustering metrics")
internal_metrics_df = comparison_df[
    [
        "Model",
        "Clusters",
        "Noise Points",
        "Silhouette Score",
        "Davies-Bouldin Index",
        "Calinski-Harabasz Index",
        "Metric Notes",
    ]
]
display(internal_metrics_df.round(4))

print("External evaluation metrics using known labels")
external_metrics_df = comparison_df[
    [
        "Model",
        "Accuracy",
        "Weighted Precision",
        "Weighted Recall",
        "Weighted F1",
        "Anomaly F1",
    ]
]
display(external_metrics_df.round(4))

plot_count_bar(
    comparison_df.set_index("Model")["Anomaly F1"],
    title="Clustering Algorithm Comparison by Anomaly F1",
    ylabel="Anomaly F1",
)

plot_count_bar(
    comparison_df.set_index("Model")["Weighted F1"],
    title="Clustering Algorithm Comparison by Weighted F1",
    ylabel="Weighted F1",
)

plot_count_bar(
    comparison_df.set_index("Model")["Silhouette Score"],
    title="Clustering Algorithm Comparison by Silhouette Score",
    ylabel="Silhouette Score",
)

plot_count_bar(
    comparison_df.set_index("Model")["Davies-Bouldin Index"],
    title="Clustering Algorithm Comparison by Davies-Bouldin Index",
    ylabel="Davies-Bouldin Index",
)
```

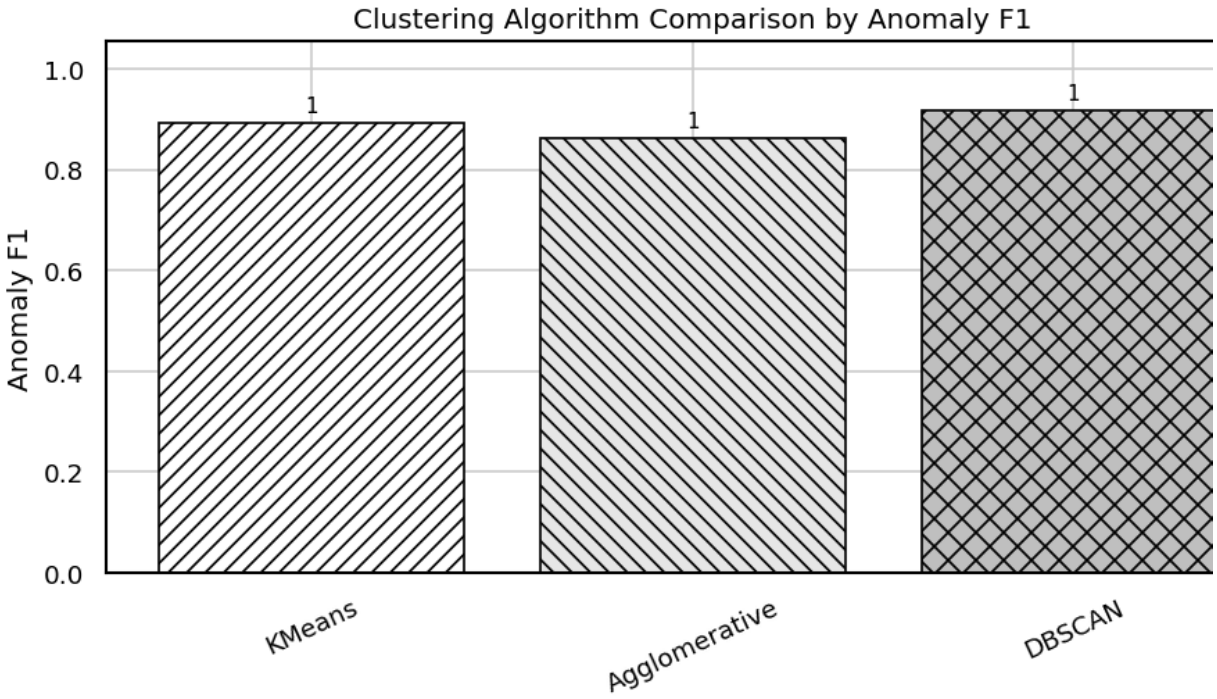
| | Model | Clusters | Noise Points | Accuracy | Weighted Precision | Weighted Recall | Weighted F1 | Anomaly F1 | Sill |
|---|---|---|---|---|---|---|---|---|---|
| **0** | KMeans | 2 | 0 | 0.9023 | 0.9161 | 0.9023 | 0.9015 | 0.8926 | |
| **1** | Agglomerative | 2 | 0 | 0.8713 | 0.8774 | 0.8713 | 0.8708 | 0.8626 | |
| **2** | DBSCAN | 48 | 623 | 0.9123 | 0.9186 | 0.9123 | 0.9120 | 0.9174 | |

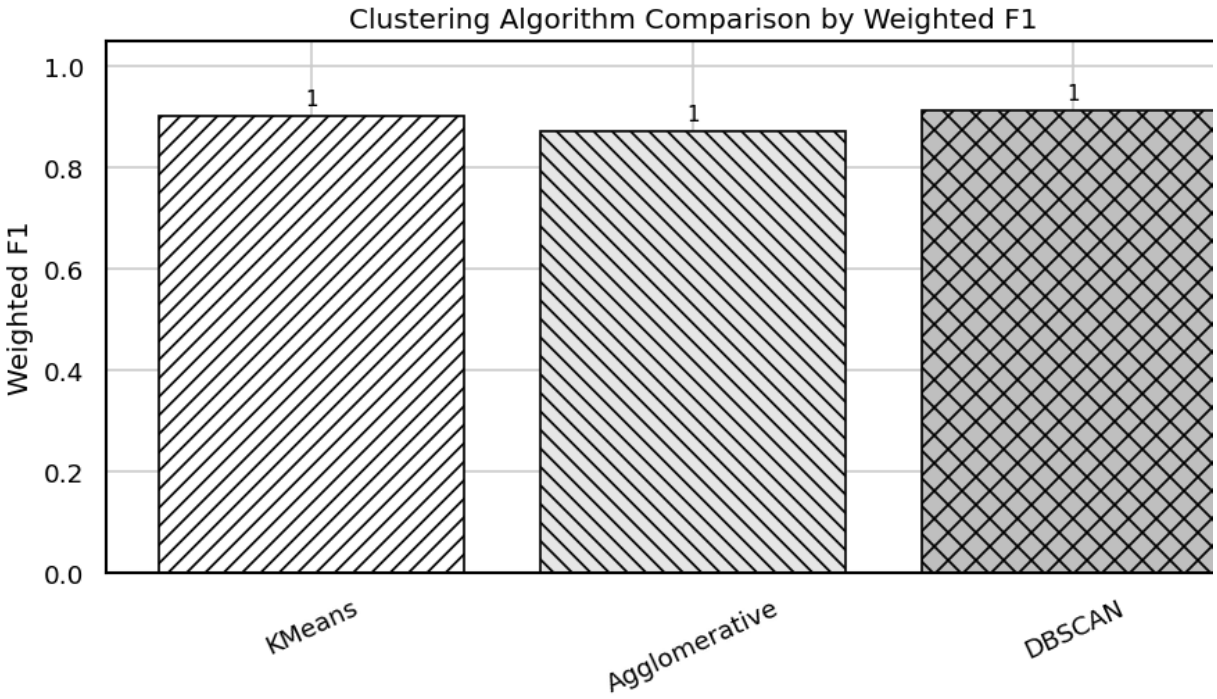
Internal clustering metrics

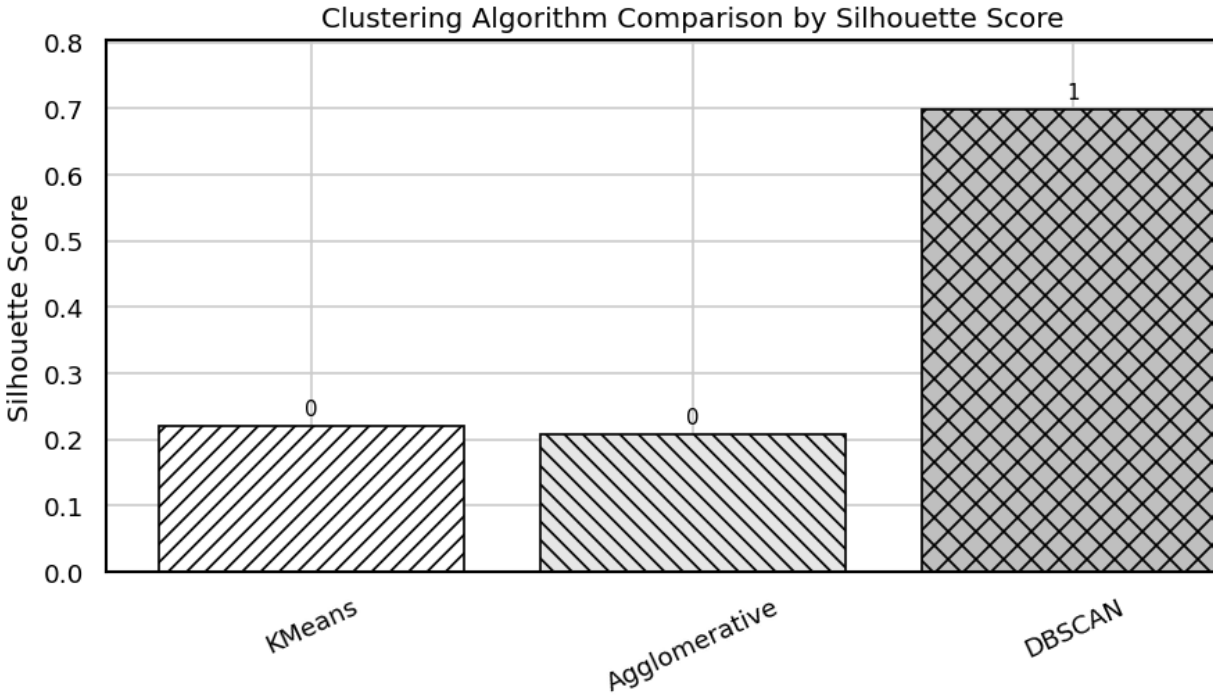
| | Model | Clusters | Noise Points | Silhouette Score | Davies-Bouldin Index | Calinski-Harabasz Index | |
|---|---|---|---|---|---|---|---|
| **0** | KMeans | 2 | 0 | 0.2203 | 2.6501 | 277.8625 | All |
| **1** | Agglomerative | 2 | 0 | 0.2074 | 2.7249 | 261.9802 | All |
| **2** | DBSCAN | 48 | 623 | 0.6983 | 0.3491 | 1399.8455 | ig |

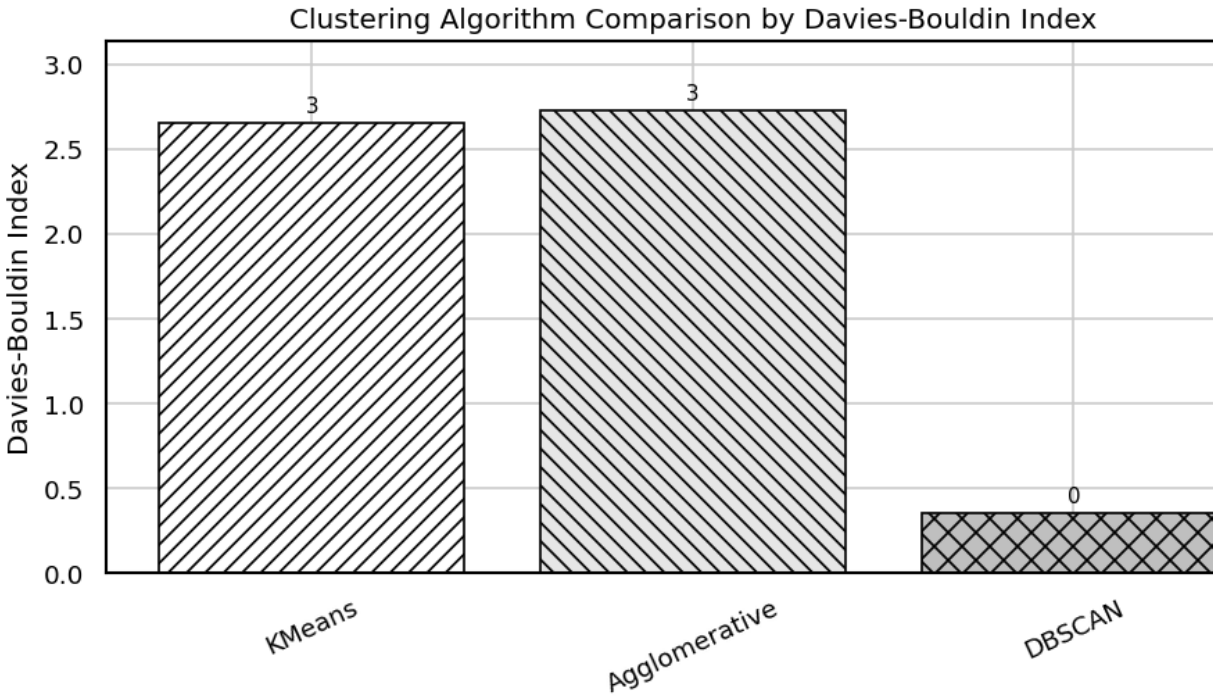
External evaluation metrics using known labels

| | Model | Accuracy | Weighted Precision | Weighted Recall | Weighted F1 | Anomaly F1 |
|---|---|---|---|---|---|---|
| **0** | KMeans | 0.9023 | 0.9161 | 0.9023 | 0.9015 | 0.8926 |
| **1** | Agglomerative | 0.8713 | 0.8774 | 0.8713 | 0.8708 | 0.8626 |
| **2** | DBSCAN | 0.9123 | 0.9186 | 0.9123 | 0.9120 | 0.9174 |




Clustering Algorithm Comparison by Weighted F1
Weighted F1
1.0
0.8
0.6
0.4
0.2
0.0
1
1
1
KMeans
Agglomerative
DBSCAN


Clustering Algorithm Comparison by Silhouette Score
Silhouette Score
0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0.0
0
0
1
KMeans
Agglomerative
DBSCAN




## 11. Final Prediction Table

In [12]:
```
prediction_table = pd.DataFrame({
    "actual": np.where(y_true == 1, "anomaly", "normal"),
    "KMeans": np.where(predictions["KMeans"] == 1, "anomaly", "normal"),
    "Agglomerative": np.where(predictions["Agglomerative"] == 1, "anomaly
    "DBSCAN": np.where(predictions["DBSCAN"] == 1, "anomaly", "normal"),
})
display(prediction_table.head(20))
```

| | actual | KMeans | Agglomerative | DBSCAN |
|---|---|---|---|---|
| **0** | normal | normal | normal | normal |
| **1** | anomaly | anomaly | anomaly | anomaly |
| **2** | normal | normal | normal | normal |
| **3** | anomaly | normal | normal | anomaly |
| **4** | normal | normal | normal | normal |
| **5** | anomaly | anomaly | anomaly | anomaly |
| **6** | anomaly | anomaly | anomaly | anomaly |
| **7** | normal | normal | normal | normal |
| **8** | anomaly | anomaly | anomaly | anomaly |
| **9** | normal | normal | normal | normal |
| **10** | normal | normal | normal | normal |
| **11** | anomaly | anomaly | anomaly | anomaly |
| **12** | anomaly | normal | normal | normal |
| **13** | normal | normal | normal | normal |
| **14** | normal | normal | normal | normal |
| **15** | anomaly | anomaly | anomaly | anomaly |
| **16** | anomaly | anomaly | anomaly | anomaly |
| **17** | anomaly | anomaly | anomaly | anomaly |
| **18** | normal | normal | normal | normal |
| **19** | normal | normal | normal | normal |